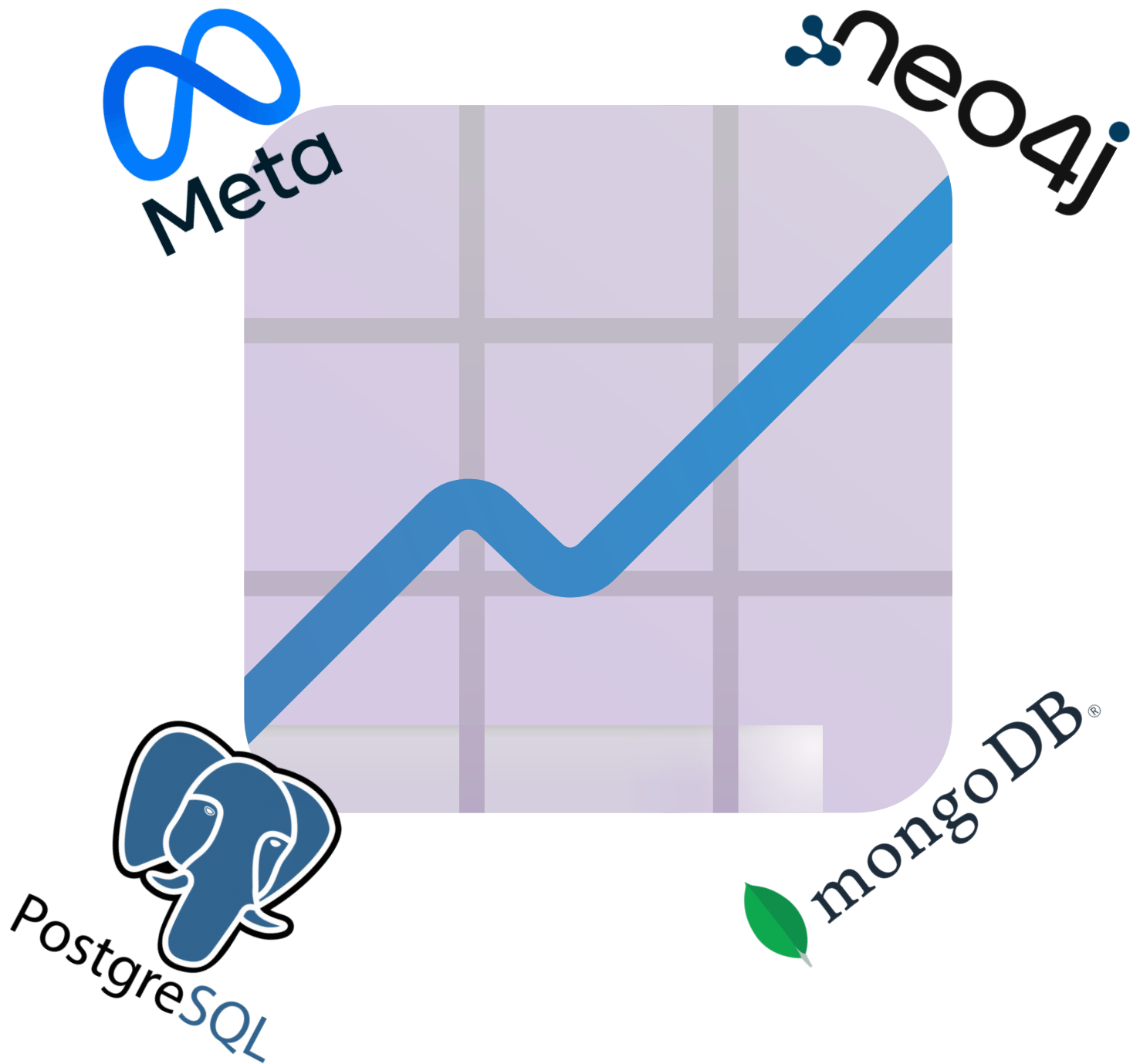


Markindex Analytics Tool



Link: <https://github.com/steffengroenandersen/kea-databases>

Group: Steffen Grøn Andersen

Date of Delivery: 08-12-2025

Introduction	2
Problem description	2
Methodology and tools	3
Case Study Application	6
Application overview	6
User stories	6
Relational database	7
Introduction to relational databases	7
Data modelling	7
Database features	11
SQL Injection	16
Applying user stories	16
Sub-conclusion	17
Document database	17
Introduction to document databases	17
Data modelling	18
Applying the user stories	19
Sub-conclusion	20
Graph database	21
Introduction to graph databases	21
Data modelling	21
Applying the user stories	23
Sub-conclusion	24
Conclusion	24
Appencides	26
Appendix A - PostgreSQL Stored Procedure	26
Appendix B - PostgreSQL Stored Function	27
Appendix C - PostgreSQL Indexes	28
Appendix D - PostgreSQL Views	29
Appendix E - PostgreSQL Triggers	30
Appendix F - PostgreSQL Transactions	31
Appendix G - PostgreSQL User Privileges	33
Appendix H - PostgreSQL Get all meta ad accounts	34
Appendix I - PostgreSQL Get all campaigns by ad account	35
Appendix J - PostgreSQL Get average campaign spend	36
Appendix K - MongoDB Document Structure	37
Appendix L - MongoDB Indexes	39
Appendix M - MongoDB Get all meta ad accounts	40
Appendix N - MongoDB Get all campaigns by ad account	41
Appendix O - MongoDB Get average campaign spend	42
Appendix P - Neo4j Get all meta ad accounts	43
Appendix Q - Neo4j Get all campaigns by ad account	44
Appendix R - Neo4j Get average campaign spend	45

Introduction

Over the past decade, digital marketing has grown increasingly complex, fueled by the rapid expansion of platforms, tools, and tracking regulations. Alongside this growth, a wide range of SaaS applications have emerged to support marketing efforts, helping businesses analyze, manage, and optimize their campaigns.

This report focuses on the development of Markindex, a tool designed to address this complexity. Markindex is a platform capable of organizing, analyzing, and presenting data in a way that simplifies decision-making for digital marketing teams.

The primary goal of the report is to explore different approaches to designing Markindex, with a particular focus on the choice and structure of database technologies. The report documents how various database systems, schemas, and data management strategies can be leveraged to create an efficient, scalable, and maintainable solution for collecting, analyzing, and presenting marketing data.

For a solo developer, the importance of selecting the right tools from the outset cannot be overstated. Given the limited resources and time constraints, it is essential to prioritize simplicity and maintainability over unnecessary complexity. Choosing appropriate technologies, frameworks, and database systems early in the development process ensures that the platform remains flexible, efficient, and easier to extend or modify as requirements evolve.

Problem description

Digital marketing is critical for e-commerce businesses as a means to engage with target audiences and drive sales. However, the field has become increasingly challenging due to several factors:

1. Tracking and privacy regulations: Changes such as Apple's App Tracking Transparency, the EU's GDPR, and national cookie laws have made it more difficult to track the customer journey from initial engagement to conversion. This complicates the measurement of campaign performance and ROI.
2. Proliferation of platforms and tools: New advertising channels, including Snapchat, Pinterest, and TikTok, have entered the market, while existing platforms continuously introduce new features and campaign types. Even email marketing platforms now offer expanded functionality.
3. Ease of entry into e-commerce: Platforms like Shopify have made it easier to launch online stores, increasing the number of small teams or solo operators responsible for managing all aspects of digital marketing.

These challenges create gaps in strategy, campaign management, and digital infrastructure. Inefficient tracking, fragmented data, and poorly optimized campaigns can lead to wasted marketing budgets and reduced profitability.

Markindex is designed to address these issues by consolidating and structuring marketing data in a way that supports actionable insights. The tool aims to explore different database technologies and design approaches to identify the most effective methods for storing, organizing, and querying marketing data. By doing so, Markindex seeks to provide a robust and flexible platform that can guide decision-making, highlight areas for improvement in paid advertising, and adapt to the evolving digital marketing landscape.

Methodology and tools

This section provides a rationale for the selection of databases, programming languages, frameworks, and development tools used in the design and development of Markindex. The primary goal of the project is to explore different database architectures while maintaining a system that is efficient, scalable, and deployable.

Methodology

The approach taken in this project focuses on creating the foundation of a larger digital marketing system without implementing all possible features. Certain aspects, such as authentication and authorization, are deliberately omitted as they do not directly relate to the evaluation of database technologies. Similarly, front-end development is not included, since the primary focus is on database design, data management, and back-end functionality.

To provide meaningful insights while keeping the scope manageable, the project concentrates on a set of select user stories that highlight key interactions with the databases. These user stories are designed to emphasize the differences between relational (PostgreSQL), document-oriented (MongoDB), and graph (Neo4j) database approaches, showcasing how each handles data structure, query complexity, and relationship modeling. By narrowing the focus in this way, the methodology ensures that comparisons remain clear and the system design remains practical for a solo developer.

Database selection

PostgreSQL

PostgreSQL is a relational database with strong ACID compliance, supporting complex queries, joins, and transactions. It is well-suited for scenarios that require structured data and advanced analytics, such as reporting on campaign performance, aggregating metrics, and maintaining data integrity across multiple tables. PostgreSQL has been chosen to represent SQL databases due to its open-source nature and my familiarity with it from previous experience, making it a practical and reliable choice for this project.

MongoDB

MongoDB is a document-oriented NoSQL database that offers a flexible schema structure. This makes it particularly suitable for handling unstructured or semi-structured marketing

data, such as user interactions, campaign metadata, and analytics logs. Its schema flexibility allows for rapid iteration of the database design as new requirements emerge. MongoDB has been chosen to represent the document databases out there because of its widespread popularity and strong community support, making it a reliable choice for projects that require scalability and adaptability.

Neo4j

Neo4j is a graph database designed for modeling and querying relationships between entities. In the context of Markindex, it can be used to represent user journeys, interactions across campaigns, and connections between different marketing channels. Graph databases allow for efficient exploration of complex relationships that would be cumbersome in relational databases. Neo4j has been chosen to represent graph databases because of its maturity, strong community support, and proven effectiveness in handling connected data, making it ideal for visualizing and analyzing intricate relationships in marketing datasets.

Database management and visualisation tools

pgAdmin

pgAdmin has been used to manage and query PostgreSQL databases, providing a user-friendly graphical interface for inspecting tables, relationships, and query results. It simplifies database administration tasks such as creating schemas, running SQL queries, monitoring performance, and managing user permissions. By offering clear visualizations of the database structure and query outputs, pgAdmin helps streamline development and debugging, making it an essential tool for efficiently managing relational data within Markindex.

MongoDB Compass

MongoDB Compass provides an intuitive graphical interface for working with MongoDB, making it easier to manage and explore document-based data. It simplifies tasks such as creating and modifying collections, inspecting and editing documents, visualizing schema structures, and running queries using a built-in query builder. Compass also offers performance insights and validation tools, helping developers quickly understand data patterns and maintain data quality. This makes it an invaluable tool for efficiently managing the flexible, unstructured datasets used in Markindex.

Neo4j Browser

Neo4j Browser, included in the Neo4j Desktop free edition, allows for interactive querying and visualization of graph data. It enables developers to explore complex relationships, run Cypher queries, and visually inspect nodes and edges, making it easier to understand the connections between entities. The tool also supports debugging and validating graph structures, providing immediate feedback on the data model. This makes Neo4j Browser an essential component for working with the graph-based datasets in Markindex and for ensuring that the relationships between marketing entities are accurately represented.

Programming languages and frameworks

JavaScript and Node.js

JavaScript and Node.js have been selected for development due to their clarity, simplicity, and widespread use. Node.js provides a versatile runtime environment for building server-side applications using JavaScript, and its non-blocking I/O model makes it well-suited for efficiently handling multiple database connections and requests. Using JavaScript across both the client and server sides also streamlines development, reduces context switching, and allows for rapid prototyping—making it an ideal choice for a solo developer exploring and comparing database solutions.

Node package manager

Node Package Manager (npm) is used to manage the libraries and dependencies required for the Node.js application. It simplifies the installation, updating, and version management of packages, allowing developers to easily incorporate third-party modules for database connectivity, data processing, and other functionalities.

Express

Express is a lightweight framework for Node.js that facilitates the rapid creation of RESTful APIs. Its simplicity and flexibility make it ideal for building the backend of Markindex, allowing the focus to remain on database exploration and data processing.

Development and deployment tools

VS Code

VS Code is a lightweight and versatile code editor that provides syntax highlighting, debugging, and extensions for JavaScript, Node.js, and database integrations, streamlining the development process.

GitHub

GitHub is used for version control and collaboration, allowing for secure code storage, tracking changes, and managing project iterations efficiently.

Docker

Docker ensures that the application and databases run consistently across different environments by containerizing the services. Docker Compose simplifies the management of multiple containers, including the API and various databases, enabling easy setup and teardown during development.

Case Study Application

Application overview

The Markindex application is designed as a data-centric platform for exploring and analyzing marketing performance data across multiple businesses. It aims to operate similarly to other SaaS applications in the marketing space, providing users with insights into campaign performance while maintaining simplicity and security.

Users can provide access tokens to their Meta advertising accounts, enabling the system to fetch advertisement data directly from these sources. Once their own marketing strategies are shared, users gain the ability to explore how other businesses advertise and analyze their results. All data exploration is conducted completely anonymously, ensuring that no sensitive information about other users is exposed.

Additionally, Markindex allows users to benchmark their own campaigns against aggregated market data. By comparing performance metrics, the system can generate a score or evaluation that helps users understand how their marketing efforts measure up against the broader market. Through these features, the application not only supports individual campaign analysis but also fosters learning from industry-wide trends and best practices.

User stories

This section outlines the core user stories implemented in Markindex, focusing on the key functionalities related to marketing data exploration and analysis. It does not cover basic operations such as signup or login, as the emphasis is on interactions that demonstrate how the system manages and presents marketing account and campaign data.

These user stories highlight the practical use cases that guide the design and evaluation of the underlying database architectures.

US001: Access multiple businesses

As a user, I want to access multiple businesses so that I can review and manage their connected marketing accounts.

US002: List all public ad accounts

As a user, I want to view a list of all public ad accounts so that I can identify which ones I want to explore further.

US003: View campaigns for a selected ad account

As a user, I want to view the campaigns belonging to a selected ad account so that I can analyze their structure and performance.

US004: View historical changes for a campaign

As a user, I want to view the historical changes of a campaign so that I can understand how its strategy has evolved over time.

US005: Benchmark own ad performance

As a user, I want to compare the performance of my own campaigns, ad sets, and ads against aggregated data from other businesses, so that I can understand how my marketing strategy measures up to the market and identify areas for improvement.

Relational database

Introduction to relational databases

Relational databases (RDBs) are a widely used type of database that organize data into tables, or relations, where each row represents a record and each column represents a specific attribute of that record. This structured approach allows for clear organization of data and enforces consistency through predefined schemas, ensuring that information adheres to a specific format. Relationships between tables—such as one-to-one, one-to-many, and many-to-many—allow for efficient representation of complex datasets while maintaining data integrity.

Data in relational databases is accessed and manipulated using Structured Query Language (SQL), which provides a powerful and standardized way to perform operations such as filtering, joining tables, aggregating metrics, and updating records. SQL enables developers to retrieve exactly the data they need and perform complex analyses, all while relying on the database to maintain consistency and accuracy.

In the context of Markindex, relational databases are particularly relevant for storing structured marketing data, such as campaigns, ad accounts, and historical performance metrics. Their schema-based design ensures that data remains consistent across multiple entities, and SQL queries allow for detailed analysis and reporting of marketing performance. This makes relational databases a strong choice for parts of Markindex where structured data and reliable querying are essential.

Data modelling

Entity Model

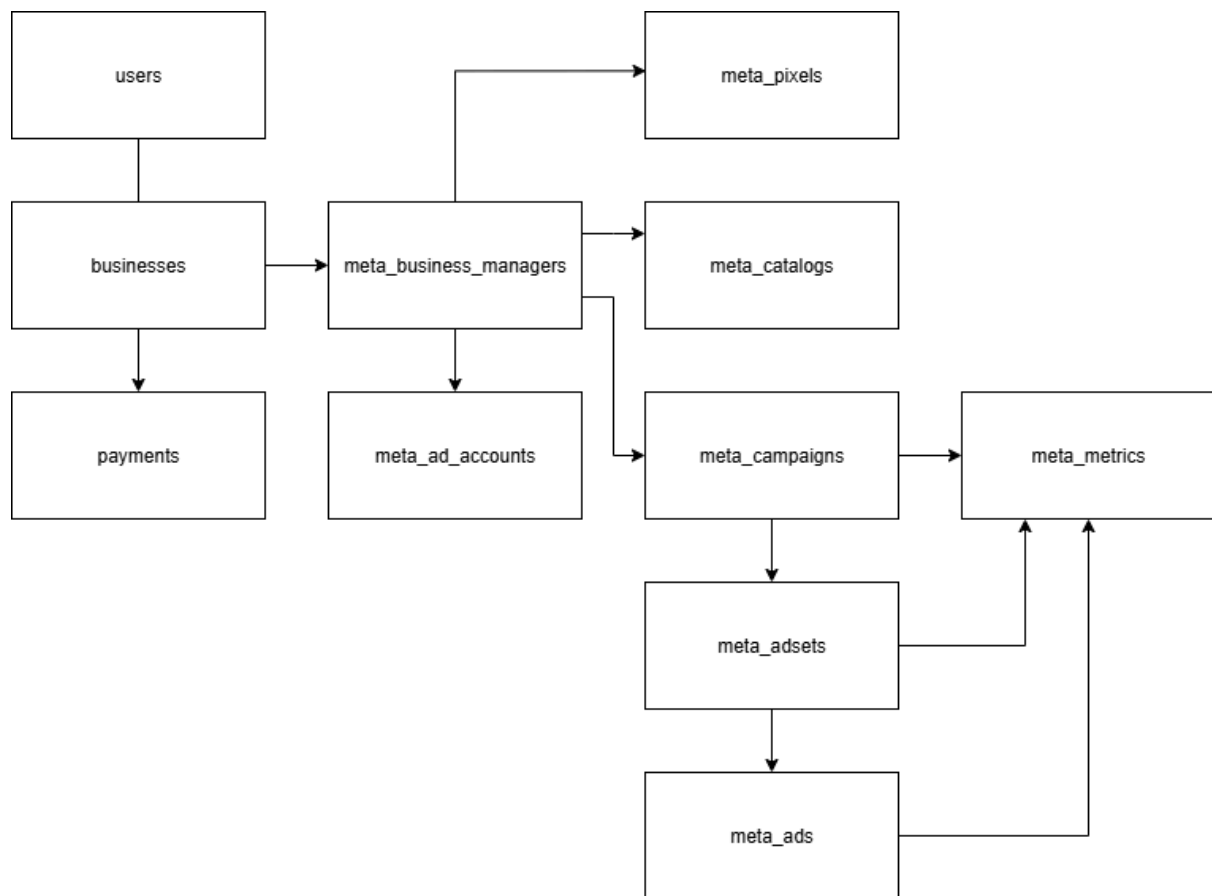
The entity model for Markindex defines the core objects and their relationships within the system, providing a foundation for both data organization and database design. At the highest level, the User entity represents individuals who interact with the platform, managing multiple Business accounts. Each business may be linked to one or more Meta Business Manager accounts, which in turn can contain multiple Meta Ad Accounts, reflecting the hierarchical structure used in Meta's advertising ecosystem.

Within each ad account, several advertising components are tracked, including Meta Pixels, which collect event data from websites, and Meta Catalogs, which store product information for dynamic ads. Marketing activities are structured through Meta Campaigns, which consist of multiple Meta Adsets, each containing individual Meta Ads. Each ad, ad set, campaign

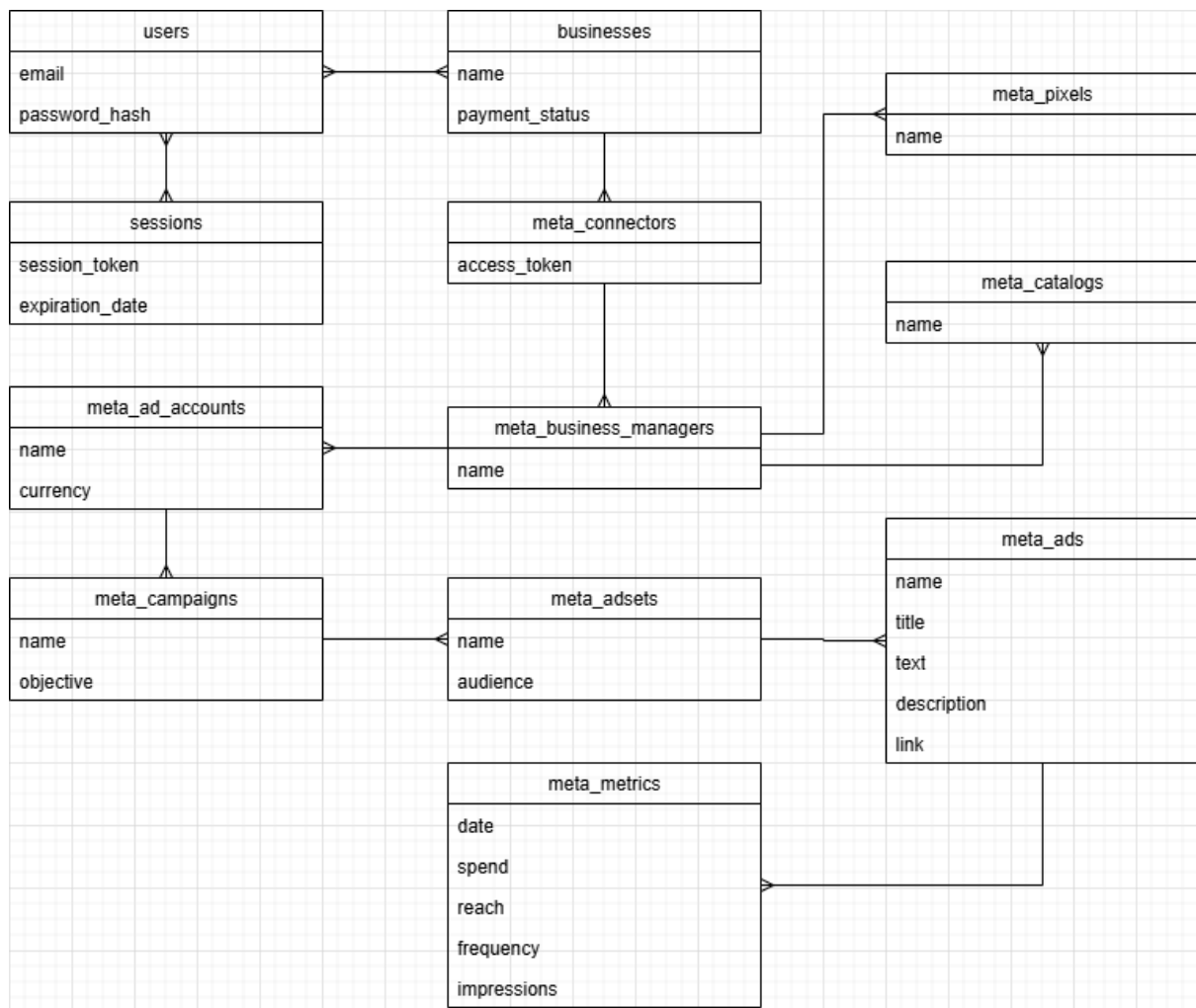
and ad account is associated with Meta Metrics, which store performance data such as impressions, clicks, conversions, and other key indicators.

This entity model allows Markindex to represent the real-world relationships between users, businesses, and advertising components, while providing a framework for organizing and querying marketing data efficiently. By defining the entities conceptually, logically, and physically, the model ensures consistency across different database implementations, supports detailed performance analysis, and enables comparisons between structured, semi-structured, and connected data approaches.

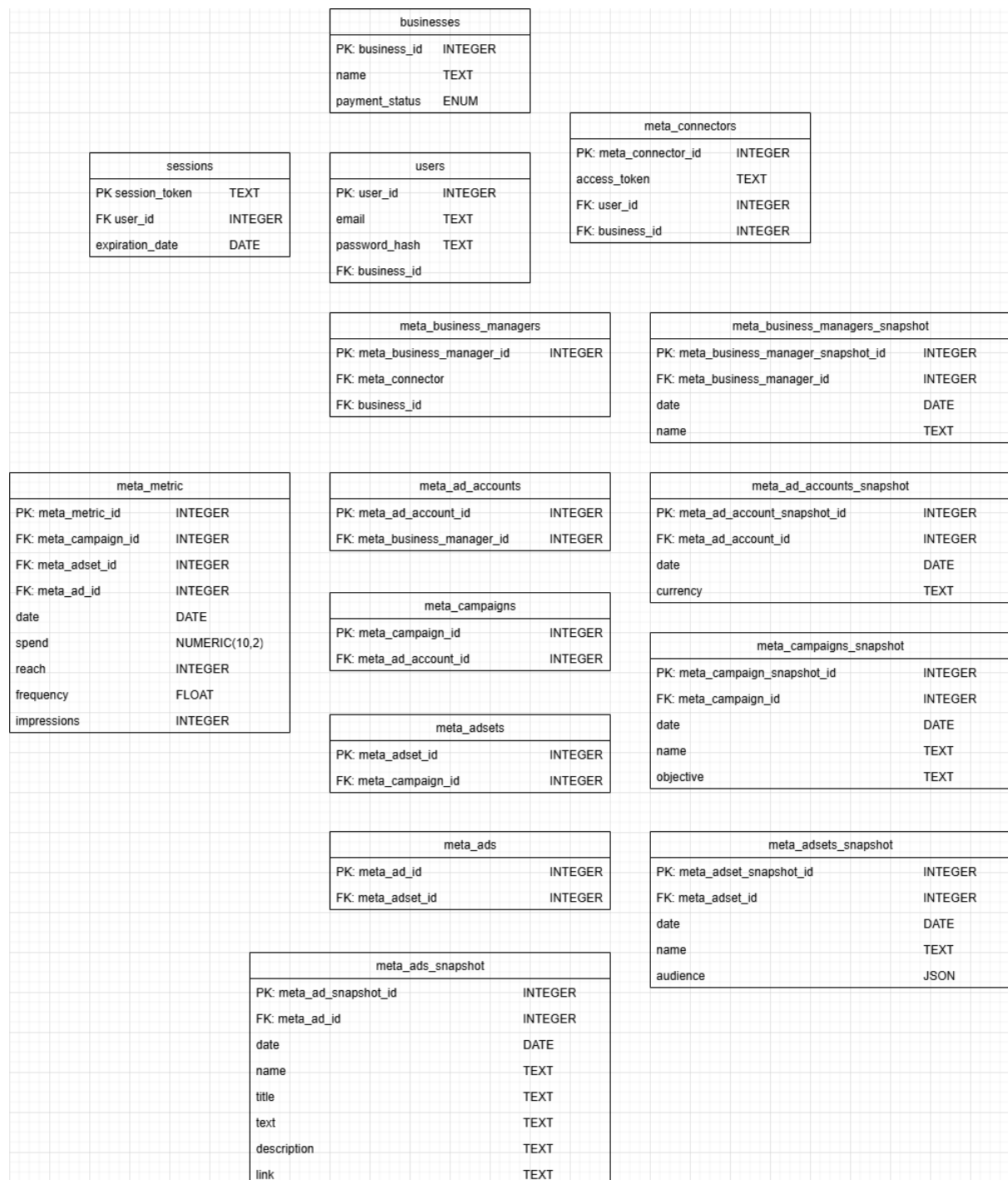
Conceptual



Logical



Physical



Normalization process

Normalization in SQL is the process of organizing data within a relational database to reduce redundancy and ensure data integrity.

It involves structuring tables and their relationships so that each table represents a single entity and each column contains atomic values. By applying standard forms—such as first, second, and third normal form—SQL databases enforce dependencies between data elements, making updates, deletions, and queries more consistent and reliable.

In the context of Markindex, normalization ensures that marketing entities like campaigns, ads, and metrics are stored efficiently, enabling accurate analysis and minimizing duplication across tables.

While normalization provides a clear structure and minimizes redundancy, it must be balanced with the format of the source data, such as the nested and semi-structured responses from the Meta API. In practice, some denormalization or temporary flattening may be necessary to efficiently ingest and process data, especially for metrics or nested objects like ad sets and ads. The goal is to maintain the benefits of normalization—consistency, integrity, and clear relationships—while adapting the schema to the realities of the data being collected, ensuring that queries remain practical and performance remains acceptable.

Database features

Data types

In the following section we will go through some of the data types chosen in this project.

SERIAL

SERIAL is an auto-incrementing integer type used for primary keys in PostgreSQL. It simplifies the creation of unique identifiers for each record, ensuring that every entity can be referenced reliably. In Markindex, SERIAL is used for IDs like `user_id`, `business_id`, and `meta_ad_account_id`. This type is ideal for primary keys because it guarantees uniqueness without requiring manual input.

INTEGER

INTEGER is a standard numeric type used to store whole numbers. It is commonly used for foreign key references, connecting tables and establishing relationships between entities. For example, `user_id` in the `user_businesses` table references the `users` table. INTEGER provides an efficient, simple way to handle numeric identifiers and small numeric values.

NUMERIC

NUMERIC is a fixed-precision decimal type used for storing precise numeric values, such as financial amounts. In Markindex, it is used for budget and spend columns to avoid rounding errors that can occur with floating-point types. The (10,2) specification allows up to 10 digits, with 2 after the decimal point, providing enough precision for marketing spend data.

TEXT

TEXT stores variable-length character data, used for emails, names, access tokens, and other textual information. While TEXT allows unlimited length, in some cases VARCHAR(n) could be a better choice, allowing you to enforce a maximum length and potentially improve storage efficiency. For example, using VARCHAR(255) for email addresses can help prevent excessively long inputs. But in the case of keeping it simple TEXT will do.

DATE

DATE stores calendar dates without time information and is suitable for columns like snapshot dates in the Markindex tables. It allows easy comparison, filtering, and sorting by date, which is important for analyzing historical campaign performance.

TIMESTAMP

TIMESTAMP stores both date and time information, providing precision for tracking events such as `last_updated` in the users table. It is useful when changes need to be recorded with exact timing, enabling accurate auditing and temporal analysis of data.

Stored procedures

A stored procedure is a precompiled set of SQL statements stored in the database that can be executed as a single call. Stored procedures help encapsulate logic, improve code reuse, and ensure consistent operations on the database.

The `insert_user` procedure provides a reusable way to add new users to the users table. It accepts an email and a password hash as input parameters and inserts a corresponding record into the database, simplifying the insertion process.

The full procedure is provided in [Appendix A](#).

Stored functions

A stored function is a database object that performs operations and returns a value. Unlike a procedure, a function always returns a result and can be used directly within queries.

The `count_all_users` function counts the total number of users in the users table and returns this value as an integer. It encapsulates the counting logic in a reusable way, allowing the value to be retrieved efficiently whenever needed.

The full function is provided in [Appendix B](#).

Index

An index is a database structure that improves the speed of data retrieval operations on a table. By creating an index, queries that search, filter, or join on the indexed column can be executed more efficiently.

The `idx_users_email` index is created on the email column of the users table, optimizing searches and lookups by email. This is particularly useful for operations such as user authentication or checking for existing accounts.

The full index definition is provided in [Appendix C](#).

Views

A view is a virtual table in a database that represents the result of a stored query. Views allow users to simplify complex queries, encapsulate business logic, and present data in a convenient format without duplicating it.

The `total_spend_all_businesses_by_date` view calculates the total advertising spend across all businesses for each date. It joins the relevant tables—businesses, Meta Business Managers, ad accounts, and metrics—aggregates the spend, and presents the results ordered by date. This view provides a quick and reusable way to analyze daily spend trends across the platform.

The full view definition is provided in [Appendix D](#).

Triggers

A trigger is a database mechanism that automatically executes a specified function in response to certain events on a table, such as INSERT, UPDATE, or DELETE. Triggers help enforce rules, maintain derived data, or audit changes without requiring manual intervention.

The `users_last_updated_trigger` automatically updates the `last_updated` timestamp of a user record whenever it is inserted or modified. It calls the `update_users_last_updated` function before each insert or update, ensuring that the `last_updated` field always reflects the most recent change.

The full trigger definition is provided in [Appendix E](#).

Events

In some database systems, an event is a scheduled action that runs automatically at specified times or intervals. Events are often used to perform routine maintenance tasks, update aggregated data, clean up old records, or generate reports without manual intervention.

For example, a marketing analytics system might use an event to refresh daily campaign metrics, calculate summary statistics, or archive historical data at midnight every day.

While PostgreSQL does not have a native event scheduler like MySQL, similar functionality can be achieved using cron jobs or external task schedulers that execute SQL scripts at regular intervals. In Markindex, no events are currently implemented, but they could be added in the future to automate tasks such as snapshot updates or metric calculations.

Constraints

Constraints are rules applied to database tables to enforce data integrity and ensure that the stored information remains accurate and consistent. They define what values are allowed in columns and how tables relate to each other, preventing invalid or inconsistent data from being entered.

Common types of constraints used in Markindex include:

Primary Key (PK): Ensures that each record in a table has a unique identifier. For example, `user_id` in the `users` table.

Foreign Key (FK): Enforces referential integrity between tables by linking a column in one table to the primary key of another. For instance, `user_id` in `user_businesses` references the `users` table.

Unique: Guarantees that the values in a column are distinct across all records, such as the `email` column in `users`.

Not Null: Ensures that certain columns cannot contain NULL values, enforcing required data entry.

By using constraints, Markindex maintains reliable relationships between users, businesses, ad accounts, and campaigns, reduces errors, and supports accurate querying and reporting of marketing data.

Transactions

A transaction in PostgreSQL is a mechanism that allows a group of SQL operations to be executed as a single, atomic unit of work. The purpose of using transactions is to ensure data integrity, consistency, and reliability. With transactions, either all operations succeed and are permanently saved (COMMIT), or none of them are applied (ROLLBACK) if an error occurs.

1. Starting the Transaction

This tells PostgreSQL that all following operations belong to one atomic block.

2. Running Through Business Entities

We retrieve all entities that need to be updated. For each entity, the system fetches new data from Meta (campaigns, adsets, ads) and attempts to insert it into the database.

3. Executing Multiple Dependent Actions

Inside the loop, we make several external API requests:

- `fetchMetaCampaigns`
- `fetchMetaAdsets`
- `fetchMetaAds`

If any of these fail (for example because Meta's API is unavailable), the error is caught and the transaction is rolled back. After fetching the data, each set is inserted into the database. However, you want to keep a transaction open for as short time as possible, so it would be more optimal to fetch the data before you open the transaction.

This approach ensures that all related updates for a business entity are applied consistently, maintaining data integrity throughout the system. The full transaction procedure is detailed in [Appendix F](#).

Auditing

Database auditing is the process of tracking and recording changes made to data, providing information about who changed what, when, and how. Auditing is important for security, debugging, compliance, data integrity, and traceability.

In PostgreSQL, a common and efficient way to implement auditing is through triggers. Triggers automatically execute a function whenever certain events occur on a table, such as INSERT, UPDATE, or DELETE. By performing auditing within the database rather than the application layer, triggers ensure that:

- Auditing cannot be bypassed by buggy or insecure applications
- All changes are logged centrally
- Consistency of audit data is maintained
- Performance impact is minimal

Although auditing has not been implemented in Markindex yet, this approach provides a clear and reliable strategy for tracking changes in future development.

Security

To manage access and maintain security within Markindex, two PostgreSQL users have been created with distinct roles and privileges.

The admin user has full access to the database, including all tables, sequences, functions, and procedures. This user can perform any operation, including creating or modifying database objects and executing routines. Default privileges are also set to ensure that the admin account automatically receives full access to any new tables, sequences, or routines created in the future. This account is intended for system administration and maintenance tasks.

The database user is restricted to standard CRUD operations (Create, Read, Update, Delete) on the existing tables and sequences. This user cannot modify database objects, execute administrative functions, or change the schema, ensuring that application operations can be performed safely without risking structural integrity. Default privileges are configured so that future tables and sequences automatically grant the same restricted access to this user.

By separating roles in this way, Markindex enforces principle of least privilege, reduces the risk of accidental or malicious schema modifications, and maintains a clear distinction between administrative and operational access.

The full user and privileges setup is provided in [Appendix G](#).

SQL Injection

SQL injection is a security vulnerability that occurs when user-provided input is inserted into a SQL query without proper validation or sanitization. This allows an attacker to alter the intended query structure—potentially retrieving, modifying, or deleting data, or even gaining full control over the database. Preventing SQL injection typically involves using prepared statements, parameterized queries, and strict input handling to ensure that user input cannot be executed as part of a SQL command.

Applying user stories

US001: Access multiple businesses

To implement US001: Access multiple businesses in PostgreSQL, the `users`, `businesses`, and `user_businesses` tables are used to model the relationships. The `user_businesses` junction table links each user to one or more businesses, enabling many-to-many access. A query can then join these tables to retrieve all businesses associated with a specific user, providing a clear list of accounts the user is authorized to manage. This approach ensures data integrity while allowing efficient querying of user access.

US002: List all public ad accounts

To support US002: List all public ad accounts, a stored function was created to retrieve every Meta Ad Account along with its most recent name. Because ad account names are stored in a snapshot table and may change over time, the function uses a LATERAL join to efficiently select the latest snapshot for each account. This ensures that the user always sees the most up-to-date information without manually managing historical records. The full function definition is included in [Appendix H](#).

US003: View campaigns for a selected ad account

To retrieve all campaigns for a specific Meta Ad Account in PostgreSQL, a stored function was created that joins each campaign with its most recent snapshot using a lateral join. This ensures the function always returns the latest campaign name while filtering by the given ad account. Results are sorted alphabetically for easy browsing. The full function definition is included in [Appendix I](#).

US004: View historical changes for a campaign

The implementation of this user story is straightforward, as it primarily involves accessing the snapshots associated with a campaign. Retrieving historical changes for a specific date simply requires filtering the snapshot array (in MongoDB) or traversing the `HAS_SNAPSHOT` relationships (in Neo4j) for the desired timestamp. Because the approach is trivial and conceptually identical across PostgreSQL, MongoDB, and Neo4j, the detailed implementation is omitted for all three databases.

US005: Benchmark own ad performance

This function computes the average spend for all Meta campaigns recorded in the `meta_metrics` table. By filtering out rows where `meta_campaign_id` is null, the query ensures that only metrics associated with valid campaigns are included in the calculation.

PostgreSQL's aggregate function `AVG()` is then applied to the `spend` column to derive the mean spend across all qualifying records. The function returns this value as a single-field result set, making it easy to integrate into reporting dashboards, analytics pipelines, or other database operations. The full PostgreSQL function definition used to generate this value is provided in [Appendix J](#).

Sub-conclusion

For Markindex's use case—where the majority of operations involve analytical aggregations across structured marketing entities with clearly defined relationships—PostgreSQL's combination of SQL expressiveness, mature optimization capabilities, materialized views, and predictable performance characteristics makes it an optimal choice for the core analytical workload. The database's ability to enforce data consistency through constraints while maintaining query efficiency through indexes ensures that marketing performance reporting remains both accurate and performant as the platform scales.

Document database

Introduction to document databases

Document databases are a type of NoSQL database that store data as flexible, self-contained documents, often in formats such as JSON or BSON. Each document can contain nested structures, arrays, and varying sets of fields, allowing for a highly adaptable schema that can evolve over time. This flexibility makes document databases well-suited for handling unstructured or semi-structured data without requiring strict, predefined schemas.

Data in document databases is accessed through query languages or APIs that can filter, project, and aggregate data within individual documents. Unlike relational databases, joins between collections are limited or avoided, with data often embedded to optimize read performance. This model enables developers to store complex hierarchical data in a single document, reducing the need for multiple related tables.

In the context of Markindex, document databases are particularly useful for fast and efficient querying of marketing data and for maintaining a flexible schema that can evolve as new fields or ad account configurations or marketing channels are introduced. However, for more complex analytical tasks—such as generating aggregated benchmarks across many advertisers or combining multiple entities for detailed reporting—the document model can become limiting. In such cases, relational databases are better suited due to their optimized support for joins, aggregations, and structured queries.

Data modelling

Entity / Document structure

The current data model for Markindex is designed to represent the hierarchical relationships between marketing entities while supporting both current and historical performance analysis. In this model, primary entities such as MetaBusinessManagers, MetaAdAccounts, MetaCampaigns, MetaAdSets, and MetaAds are stored as individual documents in a document-oriented database. Each document contains the attributes relevant to the entity and includes an embedded snapshots array to track historical changes over time.

A referenced approach is used for hierarchical relationships: campaigns reference their parent ad account, ad sets reference their campaign, and ads reference their ad set. Metrics reference all relevant parent entities (ad, ad set, campaign, ad account) to allow efficient aggregation across multiple levels. This structure supports Markindex's core functionality, including displaying all ad accounts across businesses, analyzing campaigns and ads at various levels, and calculating performance benchmarks.

The detailed structure of each document, including the embedded snapshots arrays, is provided in [Appendix K](#).

Embedded vs referenced documents

In document databases, a key design decision is whether to embed related data directly within a document or reference other documents using unique identifiers. This choice has important implications for query performance, document size, maintainability, and flexibility.

At first glance, it would have been logical to embed Meta assets (campaigns, ad sets, ads) within their parent entities due to their clear parent-child relationships. For example, campaigns belong to ad accounts, ad sets belong to campaigns, and ads belong to ad sets. Embedding these assets could simplify queries when retrieving all children for a given parent, as all the data would reside within a single document.

However, for Markindex, this approach proved problematic. Many features of the application require aggregating data across multiple ad accounts, campaigns, and businesses (e.g., benchmarking spend across all advertisers). Embedding all children would force the database to scan large nested documents, making such queries inefficient and cumbersome.

Given these considerations, hierarchical relationships between Meta assets are modeled using references. Campaigns reference their ad account, ad sets reference their campaign, and ads reference their ad set. Metrics reference all relevant parent entities, enabling efficient aggregation at any level without the overhead of nested documents.

At the same time, embedding was chosen for snapshot data. Each Meta asset document contains an embedded snapshots array that stores historical versions of the entity. Embedding snapshots within the parent document is advantageous because snapshots are read together with the parent entity, minimizing the need for additional queries and historical

tracking remains closely coupled with the current entity, simplifying the model and making it easier to implement auditing.

Index

Indexing in MongoDB is a mechanism that improves the performance of queries by allowing the database to quickly locate documents that match specific criteria. Without indexes, MongoDB must perform a collection scan, examining every document in a collection to find matching results, which can be slow for large datasets.

For example, in Markindex, campaigns are often filtered by their campaignObjective, such as "awareness", "sales", "leads", or "engagement". By creating an index on the campaignObjective field, MongoDB can quickly locate only the campaigns that match the requested objective, without scanning all documents in the MetaCampaigns collection. This ensures that queries like retrieving all campaigns for a specific objective remain fast and scalable.

An example of creating this index is provided in [Appendix L](#).

Applying the user stories

US001: Access multiple businesses

The user-to-business relationship is represented through three collections: users, businesses, and memberships. The memberships collection acts as a linking document that holds references to both userId and businessId using ObjectIDs. This structure allows each user to be associated with multiple businesses by storing references in the memberships document. To retrieve all businesses accessible to a specific user, the system queries the memberships collection using the userId and then looks up the corresponding businesses via their references. This approach ensures a flexible and scalable way to manage user access, while maintaining clear and efficient data retrieval.

US002: List all public ad accounts

To support US002 in MongoDB, an aggregation pipeline was implemented to return all Meta Ad Accounts along with their latest snapshot name. Since each document stores snapshots as an embedded array, the query extracts the most recent entry using \$arrayElemAt with an index of -1, which selects the last snapshot in the array. The result is then sorted alphabetically by name to provide a clean, user-friendly output. The full MongoDB function is included in [Appendix M](#).

US003: View campaigns for a selected ad account

To support US003: View campaigns for a selected ad account, an aggregation pipeline is used on the metaCampaigns collection. The pipeline first filters campaigns by the given metaAdAccountId, then projects each campaign's unique identifier along with the name from the most recent snapshot stored in the embedded snapshots array. Finally, the results are sorted alphabetically by campaign name. This approach leverages MongoDB's

document-oriented structure and array handling to efficiently retrieve and display all campaigns for a specific ad account, while ensuring the user sees the most up-to-date campaign information. The full function is included in [Appendix N](#).

US005: Benchmark own ad performance

This query calculates the average spend across all Meta campaigns stored in the metaMetrics collection. It begins by filtering out any records that are not linked to a valid Meta campaign, ensuring that only metrics with a defined metaCampaignId are included in the calculation. Once the dataset has been restricted to valid campaign entries, the query uses MongoDB's aggregation framework to compute the mean value of the spend field across all matching documents. The final stage formats the result by returning only the computed average, providing a clean and concise output suitable for analytics, reporting, or downstream processing. The full MongoDB aggregation pipeline used to derive this value is included in [Appendix O](#).

Sub-conclusion

For Markindex's use case—where the majority of operations involve analytical aggregations across structured marketing entities—MongoDB's document-oriented approach presents significant limitations. While the flexible schema accommodates evolving ad account configurations and the embedded snapshots array simplifies historical tracking within individual entities, the reliance on references for hierarchical relationships undermines the core advantages of document databases. Cross-collection aggregations required for benchmarking performance across campaigns, ad accounts, and businesses necessitate multiple lookups or complex aggregation pipelines, resulting in slower query performance compared to relational databases optimized for such operations. The document model's strength lies in retrieving complete entities with their nested data in single queries, but Markindex's analytical workload demands efficient joins and aggregations across multiple collections—operations for which MongoDB is inherently less suited than relational alternatives.

Graph database

Introduction to graph databases

Graph databases are designed to model and query complex relationships between entities using nodes, edges, and properties. Nodes represent entities (e.g., users, campaigns, ads), edges represent the relationships between them, and properties store relevant attributes. They are particularly powerful when applications require deep exploration of connections, pattern discovery, or path analysis across highly interconnected datasets.

Queries in graph databases use specialized languages like Cypher, which excel at traversing networks of relationships but can be more complex and less performant for straightforward aggregations or reporting tasks.

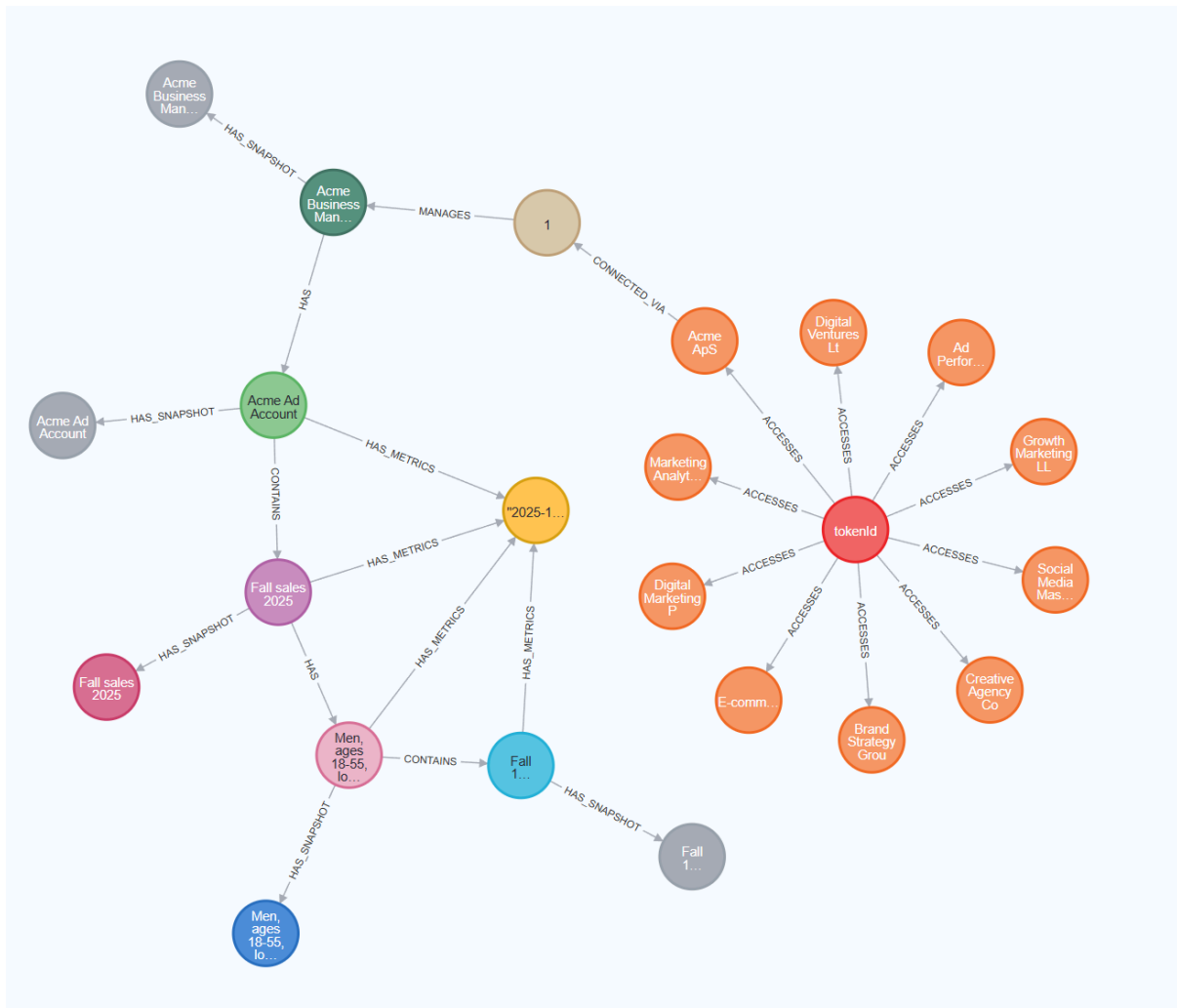
In the context of Markindex, however, graph databases are not an ideal fit. The primary operations—aggregating performance metrics across multiple advertisers to produce benchmarks and displaying other businesses' advertisement performance and settings—do not require deep relationship traversal or complex network analysis. While a graph database could represent relationships between businesses, campaigns, ad sets, and ads, the additional complexity and overhead of managing a graph database are unnecessary for the current use case. For these tasks, relational or document-oriented databases provide simpler, more efficient, and more maintainable solutions.

Interestingly, large platforms such as Meta themselves use graph databases to model social networks and their Marketing API is also structured through graph-like endpoints, which highlights their strength in highly connected data—but for Markindex's more transactional and aggregated data needs, their advantages are largely unused.

Data modelling

The Neo4j data model mirrors the hierarchical structure already present in the Meta marketing ecosystem. In the graph, a Business Manager node connects to Ad Account nodes, which link to Campaign nodes, which in turn connect to Ad Set and Ad nodes. This chain of relationships naturally reflects how Meta organises advertising assets. In a graph database, these relationships become first-class citizens rather than simply foreign keys.

This is one of the primary strengths of graph technology: the ability to explicitly model and traverse relationships. The query language Cypher allows the system to express these traversals in a declarative and intuitive way. For example, querying all campaigns belonging to a specific business becomes a matter of following the edges connecting Business Manager → AdAccount → Campaign.



In addition to the modelling of individual nodes, the Neo4j graph database relies heavily on the explicit representation of relationships. These relationships define how marketing entities are connected and make it possible to navigate the hierarchy of Meta assets in a natural and expressive way. Whereas relational and document databases imply structure through foreign keys or embedded documents, Neo4j elevates these links to first-class citizens. This enables clear, directional relationships that describe how each entity interacts with the others.

In the Markindex graph model, each relationship has been named to reflect the semantic meaning of the connection. The user-to-business layer begins with a User node that ACCESSES one or more Business nodes. A Business is then CONNECTED_VIA a MetaConnector, which represents the integration between Markindex and Meta's API.

From this connector, the hierarchy of Meta assets is represented through a chain of relationships mirroring the structure of the Meta marketing ecosystem. A MetaConnector MANAGES a MetaBusinessManager, which in turn HAS one or more MetaAdAccount nodes. Each MetaAdAccount CONTAINS MetaCampaign nodes, reflecting the campaign-level structure. A MetaCampaign then HAS MetaAdSet nodes, and each MetaAdSet CONTAINS one or more MetaAd nodes. This vertical chain captures the full lifecycle of marketing assets from business ownership to individual advertisements.

In addition to these parent–child relationships, the model introduces a cross-cutting relationship used for historical tracking. All Meta asset nodes — including accounts, campaigns, ad sets, and ads — maintain a relationship called HAS_SNAPSHOT. This relationship links each asset to a series of snapshot nodes representing stored historical states of the entity. Instead of embedding large, evolving historical documents inside each node, this structure keeps snapshots separate and connected through explicit relationships. It allows Neo4j to handle time-series history while maintaining a clean main graph.

However, even though the graph model maps neatly onto the marketing hierarchy, the practical benefits for Markindex are limited. The platform’s primary use cases revolve around performance aggregation and benchmark calculations. These operations depend more on summarising large volumes of numeric data than traversing complex relationships. While Neo4j can store and query this information, its strengths lie in exploration rather than numerical aggregation. For this reason, the graph model remains a conceptual demonstration rather than a core datastore of the system.

Applying the user stories

US001: Access multiple businesses

In Neo4j, the relationship between users and businesses is represented directly in the graph with User and Business nodes connected by an ACCESSES relationship. Each User node can have multiple outgoing ACCESSES relationships to different Business nodes, naturally modeling a many-to-many association. To retrieve all businesses a user can access, the system simply traverses the ACCESSES edges from the given User node. This graph-based approach makes querying and visualizing user access straightforward and efficient, while also allowing additional properties or metadata to be attached to the relationships if needed.

US002: List all public ad accounts

To fulfill US002 within the Neo4j graph database, a Cypher query was implemented to list all Ad Accounts along with their most recent snapshot name. Because snapshots are stored as separate nodes connected via the HAS_SNAPSHOT relationship, the query collects all snapshots for each AdAccount, orders them by date, and selects only the newest one. This approach mirrors the relational implementation but takes advantage of Neo4j’s graph traversal capabilities. The full Cypher query can be found in [Appendix P](#).

US003: View campaigns for a selected ad account

This query retrieves all campaigns associated with a selected Meta Ad Account by traversing the CONTAINS relationship between the AdAccount and Campaign nodes. By returning each campaign’s unique identifier together with its current name, the query provides a clean overview of all campaigns under that account. The results are sorted alphabetically to make it easier for users to browse or select a campaign. The full Cypher query is provided in [Appendix Q](#).

US005: Benchmark own ad performance

This query calculates the average spend across all campaigns stored in the graph database. It begins by traversing the HAS_METRICS relationship from each Campaign node to its associated DailyMetrics nodes, ensuring that only spend values tied to valid campaign metrics are included. Neo4j's avg() aggregation function is then applied to the spend property of each connected metrics node, producing the overall mean spend across all campaigns. The output is returned as a single aggregated value, providing a straightforward summary metric that can be used for reporting or comparative analysis. The full Cypher query is included in [Appendix R](#).

Sub-conclusion

For Markindex's use case—where the majority of operations involve analytical aggregations across structured marketing entities—Neo4j's graph-oriented approach introduces unnecessary complexity without delivering corresponding benefits. While the explicit modeling of relationships through nodes and edges naturally reflects the hierarchical structure of Meta's advertising ecosystem, and the Cypher query language provides intuitive syntax for traversing these connections, the platform's core functionality does not require deep relationship exploration or pattern discovery. Operations such as benchmarking campaign performance across multiple advertisers, calculating average spend metrics, and aggregating ROAS across campaign objectives are fundamentally aggregation-heavy tasks rather than traversal-heavy queries. Neo4j's strengths lie in exploring complex, interconnected networks—capabilities that remain largely unutilized in Markindex's current analytical workload. For these aggregation-centric operations, relational databases deliver superior performance and simplicity without the overhead of managing graph-specific infrastructure.

Conclusion

Based on the comprehensive analysis conducted throughout this report, PostgreSQL as a relational database emerges as the optimal choice for Markindex. While MongoDB's flexible schema offers advantages for rapidly evolving data structures and Neo4j excels at modeling complex relationship networks, neither provides sufficient benefits to outweigh the fundamental nature of Markindex's workload.

The core operations of Markindex—aggregating performance metrics across multiple advertisers, calculating benchmarks, and generating analytical reports—are inherently aggregation-heavy rather than traversal-heavy tasks. PostgreSQL's mature SQL engine, optimized for joins and aggregations, delivers superior performance for these analytical queries. Its ACID compliance ensures data integrity across the complex relationships between users, businesses, ad accounts, campaigns, and metrics, while its constraint system prevents invalid or inconsistent data entry.

Furthermore, PostgreSQL's rich feature set—including materialized views for pre-computed aggregations, sophisticated indexing strategies, stored procedures for encapsulating

business logic, and native support for transactions—provides the tools necessary to build a robust, maintainable platform. The database's ability to enforce referential integrity through foreign keys maintains the hierarchical structure of Meta's advertising ecosystem without sacrificing query efficiency.

For a solo developer with limited resources, PostgreSQL's combination of stability, predictable performance characteristics, extensive documentation, and widespread community support makes it the pragmatic choice. While the document and graph database implementations demonstrated alternative approaches to data modeling, they introduced unnecessary complexity without delivering corresponding performance or functional advantages for Markindex's analytical use case.

In conclusion, PostgreSQL's relational model aligns most effectively with Markindex's requirements, providing the optimal balance of performance, data integrity, query expressiveness, and long-term maintainability.

Appencides

Appendix A - PostgreSQL Stored Procedure

```
-- Connect to the markindex database
\connect markindex

-- Create a stored procedure to insert a user
CREATE OR REPLACE PROCEDURE insert_user(
    p_email TEXT,
    p_password_hash TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO users (email, password_hash)
    VALUES (p_email, p_password_hash);
END;
$$;
```

Appendix B - PostgreSQL Stored Function

```
-- Connect to the markindex database
\connect markindex

-- Create a stored function to count all users
CREATE OR REPLACE FUNCTION count_all_users()
RETURNS INTEGER AS $$
DECLARE
    user_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO user_count FROM users;
    RETURN user_count;
END;
$$ LANGUAGE plpgsql;
```

Appendix C - PostgreSQL Indexes

```
-- Connect to the markindex database
\connect markindex

-- Create a index for users emails
CREATE INDEX idx_users_email ON users (email);
```

Appendix D - PostgreSQL Views

```
-- Connect to the markindex database
\connect markindex

-- View: total spend across all businesses by date
CREATE OR REPLACE VIEW total_spend_all_businesses_by_date AS
SELECT
    m.date,
    SUM(m.spend) AS total_spend
FROM businesses b
JOIN meta_business_managers mbm ON mbm.business_id = b.business_id
JOIN meta_ad_accounts ma ON ma.meta_business_manager_id =
mbm.meta_business_manager_id
JOIN meta_metrics m ON m.meta_ad_account_id = ma.meta_ad_account_id
GROUP BY m.date
ORDER BY m.date;
```

Appendix E - PostgreSQL Triggers

```
-- Connect to the markindex database
\connect markindex

-- Create a trigger function for users
CREATE OR REPLACE FUNCTION update_users_last_updated()
RETURNS TRIGGER AS $$
BEGIN
    NEW.last_updated = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Add the trigger
CREATE TRIGGER users_last_updated_trigger
BEFORE INSERT OR UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_users_last_updated();
```

Appendix F - PostgreSQL Transactions

```
async function runDailyJob(req, res) {
  const client = await pool.connect();

  try {
    // 1 START TRANSACTION
    await client.query("BEGIN");

    // 2 Fetch all business entities
    const { rows: businessEntities } = await client.query(
      "SELECT * FROM BusinessEntities"
    );

    for (const entity of businessEntities) {
      console.log(`Processing entity ${entity.id}`);

      try {
        // 3 Fetch new data from Meta
        const campaigns = await fetchMetaCampaigns(entity);
        const adsets = await fetchMetaAdsets(entity);
        const ads = await fetchMetaAds(entity);

        // 4 Save to DB inside the same transaction
        // If any INSERT fails → it will be caught below, and we'll
        ROLLBACK.
        await client.query(
          "INSERT INTO Campaigns(entity_id, data) VALUES ($1, $2)",
          [entity.id, campaigns]
        );

        await client.query(
          "INSERT INTO Adsets(entity_id, data) VALUES ($1, $2)",
          [entity.id, adsets]
        );

        await client.query(
          "INSERT INTO Ads(entity_id, data) VALUES ($1, $2)",
          [entity.id, ads]
        );

      } catch (err) {
        console.error("Failure during entity processing:", err);

        // ✖ 5 ONE FAILS → rollback entire batch
      }
    }
  }
}
```



```

    await client.query("ROLLBACK");

    // You can log the failure, notify, retry later, etc.
    return res.status(500).json({
      status: "FAILED",
      reason: "Processing error on entity " + entity.id,
    });
  }
}

// 6 Everything succeeded → COMMIT
await client.query("COMMIT");
return res.json({ status: "SUCCESS" });

} catch (err) {
  // If some unexpected error happened
  console.error("Unexpected transaction error:", err);
  await client.query("ROLLBACK");
  return res.status(500).json({ status: "FAILED", error: err.message
});
} finally {
  client.release();
}
}

```

Appendix G - PostgreSQL User Privileges

```
-- Admin user
-- =====
-- Create admin user with login privileges and full access
CREATE USER admin WITH PASSWORD 'admin';

-- Grant all privileges on markindex database to admin
GRANT ALL PRIVILEGES ON DATABASE markindex TO admin;

-- Grant all privileges on all tables in public schema
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin;

-- Grant all privileges on all sequences (for auto-incrementing IDs)
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin;

-- Grant execute on all functions and procedures
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO admin;
GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA public TO admin;

-- Set default privileges for future objects
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL PRIVILEGES ON TABLES
TO admin;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL PRIVILEGES ON
SEQUENCES TO admin;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT EXECUTE ON ROUTINES TO
admin;

-- CRUD-only database user
-- =====
CREATE USER database WITH PASSWORD 'database';

GRANT CONNECT ON DATABASE markindex TO database;
GRANT USAGE ON SCHEMA public TO database;

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO
database;
GRANT USAGE, SELECT, UPDATE ON ALL SEQUENCES IN SCHEMA public TO
database;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO database;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE, SELECT, UPDATE ON SEQUENCES TO database;
```

Appendix H - PostgreSQL Get all meta ad accounts

```
CREATE OR REPLACE FUNCTION get_all_meta_ad_accounts_by_name()
RETURNS TABLE (
    meta_ad_account_id INTEGER,
    meta_ad_account_name TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        maa.meta_ad_account_id,
        maas.meta_ad_account_name
    FROM meta_ad_accounts maa
    LEFT JOIN LATERAL (
        SELECT s.meta_ad_account_name
        FROM meta_ad_accounts_snapshot s
        WHERE s.meta_ad_account_id = maa.meta_ad_account_id
        ORDER BY s.date DESC
        LIMIT 1
    ) maas ON true
    ORDER BY maas.meta_ad_account_name;
END;
$$ LANGUAGE plpgsql;
```

Appendix I - PostgreSQL Get all campaigns by ad account

```
CREATE OR REPLACE FUNCTION get_campaigns_by_ad_account(p_ad_account_id
INTEGER)
RETURNS TABLE (
    meta_campaign_id INTEGER,
    campaign_name TEXT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        mc.meta_campaign_id,
        mcs.meta_campaign_name AS campaign_name
    FROM meta_campaigns mc
    LEFT JOIN LATERAL (
        SELECT s.meta_campaign_name
        FROM meta_campaigns_snapshot s
        WHERE s.meta_campaign_id = mc.meta_campaign_id
        ORDER BY s.date DESC
        LIMIT 1
    ) mcs ON true
    WHERE mc.meta_ad_account_id = p_ad_account_id
    ORDER BY campaign_name;
END;
$$ LANGUAGE plpgsql;
```

Appendix J - PostgreSQL Get average campaign spend

```
CREATE OR REPLACE FUNCTION get_average_campaign_spend()  
RETURNS TABLE (  
    average_spend NUMERIC  
) AS $$  
BEGIN  
    RETURN QUERY  
    SELECT  
        AVG(mm.spend) AS average_spend  
    FROM meta_metrics mm  
    WHERE mm.meta_campaign_id IS NOT NULL;  
END;  
$$ LANGUAGE plpgsql;
```

Appendix K - MongoDB Document Structure

```
// User
{
  "_id": ObjectId(),
  "email": "...",
  "passwordHash": "...",
  "lastUpdated": ISODate()
}

// Business
{
  "_id": ObjectId(),
  "name": "Business Name"
}

// Membership
{
  "_id": ObjectId(),
  "userId": ObjectId("user_id"),
  "businessId": ObjectId("business_id")
}

// MetaConnector
{
  "_id": ObjectId(),
  "userId": ObjectId("user_id"),
  "businessId": ObjectId("business_id"),
  "accessToken": "..."
}

// MetaBusinessManager
{
  "_id": ObjectId(),
  "metaConnectorId": ObjectId("connector_id"),
  "businessId": ObjectId("business_id"),
  "name": "Manager Name",
  "snapshots": []
}

// MetaAdAccount
{
  "_id": ObjectId(),
  "metaBusinessManagerId": ObjectId("mbm_id"),
  "name": "Ad Account Name",
```

```

    "currency": "USD",
    "snapshots": []
}

// MetaCampaign
{
    "_id": ObjectId(),
    "metaAdAccountId": ObjectId("ad_account_id"),
    "name": "Campaign Name",
    "budget": 100.00,
    "snapshots": []
}

// MetaAdSet
{
    "_id": ObjectId(),
    "metaCampaignId": ObjectId("campaign_id"),
    "name": "AdSet Name",
    "snapshots": []
}

// MetaAd
{
    "_id": ObjectId(),
    "metaAdSetId": ObjectId("adset_id"),
    "name": "Ad Name",
    "snapshots": []
}

// MetaMetric
{
    "_id": ObjectId(),
    "adId": ObjectId("ad_id"),
    "adsetId": ObjectId("adset_id"),
    "campaignId": ObjectId("campaign_id"),
    "adAccountId": ObjectId("ad_account_id"),
    "date": ISODate(),
    "spend": 12.50,
    "impressions": 1000,
    "clicks": 50
}

```

Appendix L - MongoDB Indexes

```
db.MetaCampaigns.find({ campaignObjective: "awareness" });  
db.MetaCampaigns.find({ campaignObjective: "sales" });  
db.MetaCampaigns.find({ campaignObjective: "leads" });  
db.MetaCampaigns.find({ campaignObjective: "engagement" });
```


Appendix M - MongoDB Get all meta ad accounts

```
function getAllMetaAdAccountsByName() {  
  return db.metaAdAccounts.aggregate([  
    {  
      $project: {  
        metaAdAccountId: "$_id",  
        name: { $arrayElemAt: ["$snapshots.name", -1] }  
      }  
    },  
    {  
      $sort: { name: 1 }  
    }  
  ]).toArray();  
}
```

Appendix N - MongoDB Get all campaigns by ad account

```
function getCampaignsByAdAccount(adAccountId) {
  return db.metaCampaigns.aggregate([
    {
      $match: { metaAdAccountId: adAccountId }
    },
    {
      $project: {
        metaCampaignId: "$_id",
        campaignName: { $arrayElemAt: ["$snapshots.name", -1] }
      }
    },
    {
      $sort: { campaignName: 1 }
    }
  ]).toArray();
}
```

Appendix O - MongoDB Get average campaign spend

```
function getAverageCampaignSpend() {  
  return db.metaMetrics.aggregate([  
    {  
      $match: { metaCampaignId: { $ne: null } }  
    },  
    {  
      $group: {  
        _id: null,  
        averageSpend: { $avg: "$spend" }  
      }  
    },  
    {  
      $project: {  
        _id: 0,  
        averageSpend: 1  
      }  
    }  
  ]).toArray();  
}
```

Appendix P - Neo4j Get all meta ad accounts

```
MATCH (aa:AdAccount)
OPTIONAL MATCH (aa)-[:HAS_SNAPSHOT]->(aas:AdAccountSnapshot)
WITH aa, aas
ORDER BY aas.date DESC
WITH aa, COLLECT(aas)[0] AS latestSnapshot
RETURN aa.accountId AS metaAdAccountId,
       latestSnapshot.name AS name
ORDER BY latestSnapshot.name;
```

Appendix Q - Neo4j Get all campaigns by ad account

```
MATCH (aa:AdAccount {accountId: $adAccountId})-[:CONTAINS]->(c:Campaign)
RETURN c.campaignId AS metaCampaignId,
       c.currentName AS campaignName
ORDER BY c.currentName;
```

Appendix R - Neo4j Get average campaign spend

```
MATCH (c:Campaign)-[:HAS_METRICS]->(m:DailyMetrics)
RETURN avg(m.spend) AS averageSpend;
```